تمرین دهم معماری کامپیوتر، بخش اول

رادین شاه دائی ۴۰۱۱۰۶۰۹۶ - باربد شهرآبادی ۴۰۱۱۰۶۱۲۵

در این سوال، می‌بایست عملکرد l2cache را بررسی کنیم.

قبل از هر چیزی، ابتدا فایل benchmark.c را به گونه‌ای تغییر می‌دهیم که از N های بزرگ نیز پشتیبانی کند. این کار برای این صورت می‌گیرد که تغییرات بیشتر مشهود باشد. به منظور این کار، از فایل زیر برای benchmark استفاده می‌کنیم:

```c
// C program to multiply two square matrices.
#include <stdio.h>
#define N 100

// This function multiplies mat1[][] and mat2[][],
// and stores the result in res[][]
void multiply(int mat1[][N], int mat2[][N], int res[][N])
{
    int i, j, k;
    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            res[i][j] = 0;
            for (k = 0; k < N; k++)
                res[i][j] += mat1[i][k]*mat2[k][j];
        }
    }
}

void createMatrix(int mat[N][N]) {
    for  (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            mat[i][j] = i * j;
        }
    }
}

int main()
{
    int mat1[N][N];
    int mat2[N][N];
    createMatrix(mat1);
    createMatrix(mat2);
    int res[N][N]; // To store result
    multiply(mat1, mat2, res);
    return 0;
}
```

پس از تبدیل کد زیر به فایل قابل اجرا توسط کامپایلر gcc با optimization=1، آماده استفاده از فایل باینری هستیم.
در قدم بعد، طبق راهنمایی ها پیش‌رفته و قابلیت استفاده از سیاست‌های جایگزاری مختلف را به شبیه‌سازی با استفاده از فلگ –lru_rp اضافه می‌کنیم. سپس، دو سیاست FIFO و LRU را امتحان کرده و نتایج stats آن‌ها را بررسی می‌کنیم.

فایل stats.txt برای سیاست FIFO:

```
gem5@gem5:~/gem5$ grep l2.overallMissRate m5out/stats.txt
system.l2.overallMissRate::cpu.inst          0.991517                       # miss rate for overall accesses (Ratio)
system.l2.overallMissRate::cpu.data          0.547494                       # miss rate for overall accesses (Ratio)
system.l2.overallMissRate::total             0.598590                       # miss rate for overall accesses (Ratio)
```

فایل stats.txt برای سیاست LRU:

```
gem5@gem5:~/gem5$ grep l2.overallMissRate m5out/stats.txt
system.l2.overallMissRate::cpu.inst          0.991517                       # miss rate for overall accesses (Ratio)
system.l2.overallMissRate::cpu.data          0.543572                       # miss rate for overall accesses (Ratio)
system.l2.overallMissRate::total             0.595119                       # miss rate for overall accesses (Ratio)
```

همانطور که مشاهده می‌شود، تفاوت چندانی میان این دو سیاست حتی در N=100 نیز وجود ندارد.

پس از این مرحله، با استفاده از راهنمایی‌های درون صورت سوال فایل csv دسترسی به حافظه را پیدا می‌کنیم. سپس این فایل را به سیستم خود انتقال داده تا الگوریتم belady را روی آن اعمال کنیم.

توضیح این الگوریتم: جناب belady در سال ۱۹۶۶ در مقاله‌ای ثابت می‌کند که سیاست بهینه برای جایگزاری یک line از کش، این است که ببینیم کدام آدرس درون آن set، دیرتر از بقیه بدان در آینده دسترسی یافته می‌شود. با استفاده از یافته‌های آقای belady، به سادگی برنامه زیر را برای پیدا کردن optimal miss rate ارائه می‌دهیم (عکس توابع مورد استفاده شده در صفحه بعد آمده‌است.)

تابع simulate_cache، مقدار hit_rate بهینه را بر میگرداند. پس از اجرای این برنامه، مقدار زیر برای optimal miss rate برای N=100 بدست می‌آید:

```
52    print(1 - simulate_cache(df))

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

shahdaei@Radins-MBP test % python3 optimal.py
0.3723193753676667
```

```
def get_tag(address):
    return (address // line_size) // num_sets

def get_index(address):
    return (address // line_size) % num_sets

def simulate_cache(df):
    cache = {i: [] for i in range(num_sets)}
    access_sequence = df['Address'].tolist()
    hit_count = 0
    total_access = len(access_sequence)

    for i, address in enumerate(access_sequence):
        address_index = get_index(address)
        address_tag = get_tag(address)

        if address_tag in cache[address_index]:
            hit_count += 1
            continue

        if len(cache[address_index]) < num_ways:
            cache[address_index].append(address_tag)
            continue

        future_use = []
        for tag in cache[address_index]:
            future_address = total_access
            for j in range(i+1, total_access):
                address_in_cache = tag * num_sets + address_index
                if (access_sequence[j] // 128 == address_in_cache):
                    future_address = j
                    break
            future_use.append(future_address)


        cache[address_index][future_use.index(max(future_use))] = address_tag

    return hit_count / total_access
```